



CSA0501-DATABASE MANAGEMENT SYSTEMS

LAB EXPERIMENTS

DATE: 7/08/2026

EXPERIMENT 17 -- HIGH LEVEL PROGRAMMING EXTENSIONS-FUNCTIONS

AIM:

To implement **user-defined functions and recursive procedures** using MySQL.

PROCEDURE:

1. Select the database containing the student_cgpa table.
2. Create a function using CREATE FUNCTION.
3. Use the IF statement to classify students based on their CGPA.
4. Execute the function using SELECT.
5. Create a recursive procedure to calculate the factorial of a number.
6. Call the recursive procedure and display the result.

PROGRAM 1:

Create a function that returns the level of a student based on CGPA.

- CGPA > 9.00 → EXCELLENT
- CGPA >= 8.00 AND <= 9.00 → GOOD
- CGPA < 8.00 → AVERAGE

1. CREATE TABLE

```
CREATE TABLE student_cgpa (  
  reg_no VARCHAR(15) PRIMARY KEY,  
  student_name VARCHAR(50) NOT NULL,  
  department VARCHAR(20),  
  no_of_subs_pass INT,  
  no_of_subs_fail INT,  
  cgpa DECIMAL(3,2)  
);
```

Then Insert Values:

```
INSERT INTO student_cgpa  
(reg_no, student_name, department, no_of_subs_pass, no_of_subs_fail, cgpa)  
VALUES  
( '192411123', 'Deepthi', 'CSE', 22, 2, 8.71),  
( '192512121', 'Varun', 'IT', 27, 1, 9.23),  
( '192312117', 'Aadhya', 'ECE', 25, 3, 8.97),  
( '192411227', 'Dimple', 'CSE', 31, 0, 9.37),  
( '192820896', 'Mohan', 'EEE', 18, 9, 6.48),  
( '192411122', 'Yogitha', 'CSE', 31, 1, 9.01),  
( '192516871', 'Sadhya', 'IT', 25, 5, 7.22),
```

```
(192111101,'Rahul','MECH',30,12,5.14),
(192411010,'Alekhyia','CSE',35,2,9.01),
(192511135,'Rahul','CIVIL',37,0,9.72);
```

```
SELECT * FROM student_cgpa;
```

OUTPUT:

The screenshot shows a database management tool interface. At the top, there's a 'Result Grid' with a table of student data. Below it, the 'Action Output' pane shows the execution of SQL commands.

reg_no	student_name	department	no_of_subs_pass	no_of_subs_fail	cgpa
192111101	Rahul	MECH	30	12	5.14
192312117	Aadhya	ECE	25	3	8.97
192411010	Alekhyia	CSE	35	2	9.01
192411122	Yogitha	CSE	31	1	9.01
192411123	Deepthi	CSE	22	2	8.71
192411227	Dimple	CSE	31	0	9.37
192511135	Rahul	CIVIL	37	0	9.72
192512121	Varun	IT	27	1	9.23
192516871	Sadhya	IT	25	5	7.22
192820896	Mohan	EEE	18	9	6.48

#	Time	Action	Message	Duration / Fetch
1	22:41:47	CREATE TABLE student_cgpa (reg_no VARCHAR(15) PRIMARY KEY, student_name VARCHAR(50) NO...	0 row(s) affected	0.031 sec
2	22:44:31	INSERT INTO student_cgpa (reg_no, student_name, department, no_of_subs_pass, no_of_subs_fail, cgpa) VA...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.000 sec
3	22:46:18	SELECT * FROM student_cgpa LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec

2. PROGRAM

```
DELIMITER //
```

```
CREATE FUNCTION GetStudentLevel(student_cgpa DECIMAL(3,2))
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
```

```
    DECLARE student_level VARCHAR(20);
```

```
    IF student_cgpa > 9.00 THEN
        SET student_level = 'EXCELLENT';
```

```
    ELSEIF student_cgpa >= 8.00 AND student_cgpa <= 9.00 THEN
        SET student_level = 'GOOD';
```

```
    ELSE
        SET student_level = 'AVERAGE';
    END IF;
```

```
    RETURN student_level;
END //
```

```
DELIMITER ;
```

```
SELECT student_name, cgpa,
       GetStudentLevel(cgpa) AS student_level
FROM student_cgpa;
```

OUTPUT:

Result Grid			 Filter Rows:
	student_name	cgpa	student_level
▶	Rahul	5.14	AVERAGE
	Aadhya	8.97	GOOD
	Alekhya	9.01	EXCELLENT
	Yogitha	9.01	EXCELLENT
	Deepthi	8.71	GOOD
	Dimple	9.37	EXCELLENT
	Rahul	9.72	EXCELLENT
	Varun	9.23	EXCELLENT
	Sadhya	7.22	AVERAGE
	Kavya	8.65	GOOD
	Mohan	6.48	AVERAGE

PROGRAM 2.

Write a recursive MySQL procedure to compute the factorial of a number.

PROCEDURE:

1. Create a procedure with an input number.
2. Check whether the number is greater than 1.
3. If true, recursively call the procedure with n - 1.
4. Multiply the result by n.
5. Continue until n becomes 1.
6. Display the factorial.

PROGRAM:

```
SET SESSION max_sp_recursion_depth = 255;
DELIMITER //
```

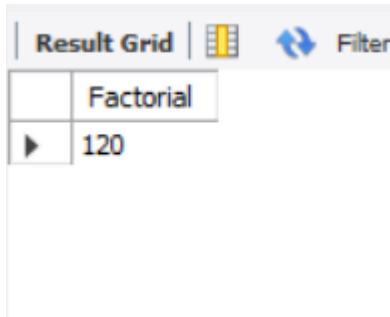
```
CREATE PROCEDURE factorial(
    IN n INT,
    OUT result BIGINT
)
BEGIN
    DECLARE temp BIGINT;

    IF n <= 1 THEN
        SET result = 1;
    ELSE
        CALL factorial(n - 1, temp);
        SET result = n * temp;
    END IF;
END //
```

```
DELIMITER ;
```

```
CALL factorial(5, @result);  
  
SELECT @result AS Factorial;
```

OUTPUT:



The screenshot shows a MySQL Result Grid window. At the top, there is a tab labeled 'Result Grid' with a yellow icon, a blue double-headed arrow icon, and a 'Filter' button. Below the header, there is a table with one column named 'Factorial'. The first row of data shows the value '120'.

	Factorial
▶	120

RESULT: Thus the Functions using program in MySQL executed successfully .